DESIGN AND DEVELOPMENT OF A LINUX-BASED ALARM CLOCK AND TIME MANAGEMENT DASHBOARD

A Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Submitted by

V. Hari Priya | 2520080015

M. Sai Harika | 2520080068

Under the Esteemed Guidance of

Dr Dr. Ragupathi M

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

We hereby declare that the project report entitled **"CHRONOS: ALARM CLOCK AND TIME MANAGEMENT DASHBOARD"** is submitted by **V. HARI PRIYA (2520080015) AND M. SAI HARIKA (2520080068)** in partial fulfillment of the requirements for the Bachelor of Technology degree during 2026-2027. The report describes the Chronos implementation prepared for this project under the guidance of Dr. Ragupathi M. The implementation and results stated here correspond to the project files and tests available at the time of preparation.

V. Hari Priya - 2520080015

M. Sai Harika - 2520080068

K L (Deemed to be) University

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled "**CHRONOS: ALARM CLOCK AND TIME MANAGEMENT DASHBOARD**" has been prepared and submitted by **V. HARI PRIYA (2520080015) AND M. SAI HARIKA (2520080068)** during the academic year 2026-2027, under the guidance of Dr. Ragupathi M, in partial fulfillment of the requirements for the award of the **BACHELOR OF TECHNOLOGY DEGREE** in Department of Artificial Intelligence and Data Science & Department of Computer Science and Information Technology.

Signature of Guide

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who have assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. Kotte Anusha**, and my guide **Dr. Ragupathi M**, Department of Computer Science and Information Technology which cannot be expressed in words for his tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr Sreenath Itikela(AI&DS) / Dr. Kaja Shareef (CSIT)**, Head of the ***Department for Artificial Intelligence and Data Science / Department for Computer Science and Information Technology*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

V. Hari Priya – 2520080015

M. Sai Harika – 2520080068

ABSTRACT

Chronos is a local alarm-clock and time-management dashboard built to combine a usable browser interface with an educational Linux systems program. Its native C/POSIX service serves the web page and JSON API on loopback, while a separate child process created with fork() checks alarm deadlines. Parent and child share alarm records through a process-shared memory mapping protected by a semaphore. SIGALRM provides the scheduler tick, and a pipe communicates fired events to the parent for persistence.

The dashboard supports one-time alarms, weekly alarms selected by weekday, pause/resume, five-minute snooze and alarm history. The browser also provides a timer, a lap-capable stopwatch, a selectable world-clock map and a repeating audio alert with visible stop/snooze controls. Alarm definitions and history are stored as tab-separated files. The application is designed for local Ubuntu or Ubuntu on WSL 2.

The C program was clean-built with GCC and validated with a Python HTTP smoke test. Integration checks covered static assets, alarm creation, scheduler firing, weekly recurrence, snooze, restart persistence and deletion of individual history records.

Keywords: POSIX, fork, SIGALRM, shared memory, semaphore, alarm scheduler, Ubuntu, browser dashboard.

INDEX

S. No.	Chapter	Topics	Page
1	Introduction	Chapter 1 topic summary	9
2	System Requirements	Chapter 2 topic summary	11
3	Technology Stack	Chapter 3 topic summary	12
4	System Architecture	Chapter 4 topic summary	13
5	Design	Chapter 5 topic summary	14
6	Implementation	Chapter 6 topic summary	16
7	Features	Chapter 7 topic summary	17
8	Testing	Chapter 8 topic summary	18
9	Deployment	Chapter 9 topic summary	20
10	Challenges And Limitations	Chapter 10 topic summary	21
11	Future Enhancements	Chapter 11 topic summary	22
12	Conclusion	Chapter 12 topic summary	23
13	References	Chapter 13 topic summary	24
14	Appendices	Chapter 14 topic summary	25
	Acknowledgement	Academic acknowledgement	4
	Abstract	Project summary	6

LIST OF FIGURES, TABLES AND ABBREVIATIONS

Figure 4.1 Chronos process and web architecture

Figure 5.1 Logical alarm and history data model

Figure 14.1 Chronos dashboard overview

Figure 14.2 World-clock map and city cards

Figure 14.3 Alarm creation and weekday selection

Table 2.1 Software requirements; Table 3.1 Technology stack

Table 5.1 Alarm fields; Table 5.2 HTTP API

Table 8.1 Integration-test results

Abbreviation	Meaning
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
IANA	Internet Assigned Numbers Authority
IPC	Inter-Process Communication
JSON	JavaScript Object Notation
POSIX	Portable Operating System Interface
TSV	Tab-Separated Values
WSL	Windows Subsystem for Linux

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Clocks, alarms, timers and stopwatches are everyday tools, but they are often scattered across devices and applications. Chronos brings these time-management functions into one browser dashboard while keeping alarm scheduling in a native Linux service. The project is also an operating-systems study: its scheduler is a separate process created with fork(), coordinated through shared memory and a process-shared semaphore, and signalled with POSIX signals.

The browser provides a responsive pink-themed interface for local time, alarms, countdown timing, stopwatch laps and world clocks. A small C server on Ubuntu serves that interface and its JSON API from localhost. This design allows the same project to demonstrate operating-system concepts while remaining convenient to use from a browser.

1.2 Problem Statement

A browser-only alarm can stop running when a tab closes, while a command-line-only scheduler is inconvenient for routine use. A disconnected collection of tools also provides no single place to create, review, snooze or remove alarms and their history. The project addresses these usability gaps with a local web interface connected to a Linux process-based scheduler.

1.3 Scope of the Project

Chronos is a local development and academic project, not a public alarm service. The C parent serves a small set of HTTP routes and static assets; its forked child checks enabled alarms once per second. Alarms and firing history are persisted as tab-separated files. The browser handles the timer, stopwatch, world-clock formatting, alarm tone and optional desktop notifications.

- One-time alarms and weekly alarms selected by weekday.
- Pause, resume, delete, snooze and history-management controls.
- Browser timer and stopwatch with lap capture.
- World clocks using browser-supported IANA time-zone identifiers.
- Ubuntu/WSL build, run and smoke-test workflow in VS Code.

1.4 Objectives

- Demonstrate fork(), POSIX signals, shared memory, semaphores and parent-child communication.
- Serve a usable browser dashboard and JSON API from a local C application.
- Support persistent one-time and recurring alarms, snooze and firing history.
- Provide timer, stopwatch, time-zone clocks and responsive UI in the browser.
- Validate the native service with repeatable integration smoke tests.

1.5 Importance of the Application

The system connects visible user actions to concrete operating-system services: process creation, signals, synchronization, pipes, file persistence and sockets. Students can inspect these connections in the source code instead of treating alarms as a purely visual feature. Local-only networking also keeps the deployment easy to run and understand.

1.6 Target Users and Audience

The intended audience is students learning Linux systems programming, project evaluators, and users who want a simple local clock dashboard. The report therefore describes both the user workflow and the C/POSIX implementation behind it.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application is lightweight and does not require specialised hardware. A dual-core processor, 4 GB of memory and approximately 100 MB of free project space are sufficient for development and local use. A display, keyboard, mouse or touch input, speakers or headphones (for browser sound), and a network loopback interface complete the setup.

2.2 Software Requirements

- Ubuntu Linux or Ubuntu running through WSL 2.
- GCC with C11/POSIX headers, GNU Make and Python 3 for smoke tests.
- Visual Studio Code with the WSL and C/C++ extensions for the recommended Windows workflow.
- A modern browser with JavaScript, Fetch, Web Audio and Intl time-zone support.
- No external database server, package installation, cloud account or runtime network service is required.

Component	Recommended baseline
CPU and memory	Dual-core CPU; 4 GB RAM
Operating system	Ubuntu Linux / Ubuntu on WSL 2
Compiler / build	GCC with C11/POSIX headers; GNU Make
Browser	Recent standards-compliant desktop browser
Disk space	Approximately 100 MB for project and build

2.3 Development Tools

The C service is compiled using the Makefile. VS Code workspace configuration supplies Ubuntu GCC IntelliSense, a build task and a Run and Debug profile that compiles before launching the server. Python's standard-library HTTP client is used by tests/smoke_test.py. The project does not depend on a JavaScript package manager or a third-party front-end framework.

2.4 Runtime and Operational Requirements

The Ubuntu service must remain running for alarms to be scheduled. The computer must remain awake; the application does not prevent system suspend. A browser tab must remain open to play browser-generated audio or show an optional desktop notification. Scheduling and persistence are performed by the native service even when the dashboard tab is closed, but sound is not emitted by the C process itself.

CHAPTER 3: TECHNOLOGY STACK

3.1 Front-End Technologies

The interface uses semantic HTML5, CSS and vanilla JavaScript. Responsive CSS adapts navigation and cards to small screens. JavaScript updates clocks using Intl.DateTimeFormat, manages timer and stopwatch state, calls the same-origin JSON API using Fetch, and generates a repeating tone through the Web Audio API. The browser Notification API is optional and requires permission.

Layer	Technology	Purpose
Interface	HTML5, CSS, vanilla JS	Responsive UI and state
Native service	C11 and POSIX APIs	HTTP, scheduling and IPC
Persistence	TSV files	Alarm definitions and history
Build / test	Make, GCC, Python 3	Compile and smoke test

3.2 Back-End Technologies

The backend is a C11/POSIX program. It uses Berkeley/POSIX sockets and a poll-based event loop for local HTTP requests; fork() creates the independent alarm scheduler. mmap(MAP_SHARED) and sem_init(..., pshared=1, ...) provide shared alarm state protected by a process-shared semaphore. SIGALRM drives a one-second scheduler tick. A pipe carries alarm-fired events back to the parent.

3.3 Storage

There is no SQL or NoSQL database. The parent stores alarm records in data/alarms.tsv and firing events in data/history.tsv. The scheduler's live alarm records reside in shared memory; the parent persists state when operations change it. ALARM_DATA_DIR can direct runtime files to another directory. Runtime TSV files are excluded from source-control tracking.

3.4 Development and Version Control

GNU Make coordinates compilation and test execution. VS Code tasks and launch configuration support direct Ubuntu/WSL development. The workspace may be placed in a Git repository, but the supplied project directory itself was not assumed to be a Git repository when this report was prepared.

3.5 APIs and External Services

The browser communicates only with the local Chronos HTTP API. The front end does not call a remote time, notification, map-tile or authentication service. World-clock values come from the browser's built-in time-zone database, and the illustrated map is a local SVG asset in the HTML.

CHAPTER 4: SYSTEM ARCHITECTURE

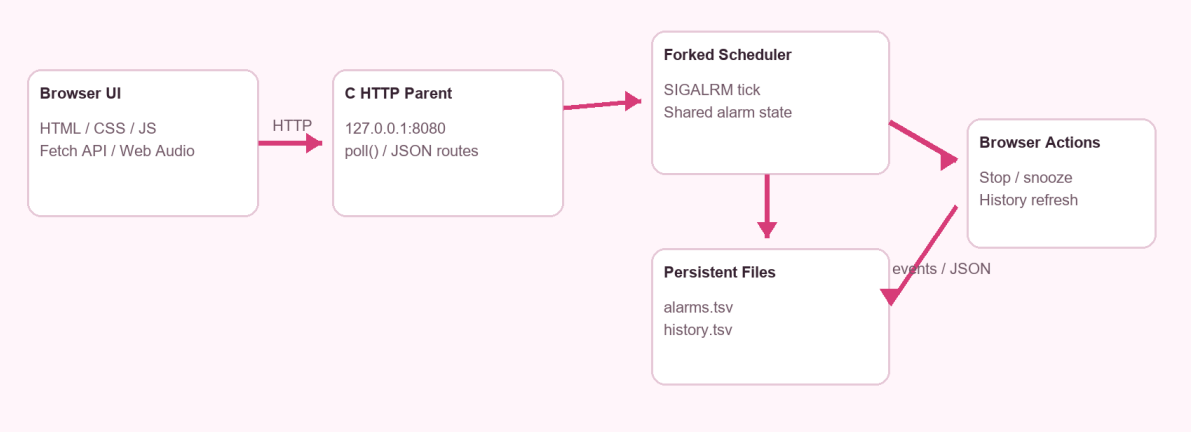


Figure 4.1: Chronos process and web architecture.

4.1 High-Level Architecture

Figure 4.1 shows the two execution environments. The browser renders the UI and calls the parent HTTP process. The parent serves static assets, validates API requests, updates shared alarm data and records event history. The forked child checks alarm due times; it sends a compact fired-event record to the parent through a pipe.

4.2 Process and Communication Design

The parent maps a SharedState object with MAP_SHARED before forking. Both processes inherit the same mapping and use an unnamed semaphore configured for process sharing. The scheduler blocks SIGALRM except while waiting with sigsuspend(), avoiding a polling delay while retaining a periodic one-second check. The parent monitors its listening socket and the scheduler event pipe with poll().

Alarm mutations and the scheduler inspection use the shared semaphore. When an alarm becomes due, the child updates its live state and writes a FiredAlarm structure to the pipe. The parent appends the event to history.tsv, updates the persisted alarm file, and the browser discovers the new history record during its periodic state refresh.

4.3 Deployment Architecture

The service binds to IPv4 loopback at 127.0.0.1, by default on port 8080. The browser loads /, /style.css and /app.js from this same server; requests to /api/state, /api/alarms and /api/history use the same origin. On Windows, WSL localhost forwarding allows a Windows browser to access the Ubuntu service. There is no cloud deployment or CI/CD pipeline in the current project.

CHAPTER 5: DESIGN



Figure 5.1: Logical alarm and history data model.

5.1 User-Interface Design

The dashboard is arranged as a compact time-first workspace. The overview shows local time; the alarm section creates and manages schedules; the two time-tool panels contain the browser timer and stopwatch; the world-clock map adds city cards; and recent activity lists fired alarms. A dialog collects the alarm label, local date/time and zero or more repeat weekdays.

The interface prioritises visible status and direct recovery actions. When a timer or alarm rings, an alert panel offers Stop sound and, for clock alarms, Snooze 5 min. Each fired history row has its own delete control. On smaller screens the side navigation becomes a horizontal strip and cards reflow.

5.2 Alarm and History Data Model

Figure 5.1 presents the logical records. An alarm has an integer ID, a label, an enabled flag, a scheduled Unix timestamp, a seven-bit repeat-day mask, an IANA time-zone name and an optional snooze timestamp. Sunday is bit 0 and Saturday is bit 6. A zero mask represents a one-time alarm. History rows record an alarm ID, the occurrence timestamp and a label snapshot; the same recurring alarm may therefore produce multiple history rows.

Alarm field	Meaning
id / label	Stable alarm identity and visible name
when / enabled	Next scheduled timestamp and active flag
repeat_days	Seven-bit Sunday-to-Saturday mask
timezone	IANA zone for local weekday recurrence
snooze_until	Temporary snooze deadline

5.3 Persistence Format

The alarm TSV columns are: id, scheduled timestamp, enabled flag, repeat-day mask, snooze-until timestamp, time-zone identifier and label. History rows use: occurrence timestamp, alarm ID and label. Labels are sanitised against tabs and newlines before storage. Existing three-field alarm records are read as legacy one-time records; newer fields are saved in the current format.

5.4 API Design

The API is deliberately small and JSON-based. IDs and occurrence timestamps identify individual alarm and history actions. A delete-history request removes only the history occurrence; it does not delete the scheduled alarm. HTTP error responses identify invalid input, missing records and conflicting operations.

Method	Route	Purpose
GET	/api/state	Current alarm definitions and latest 20 history rows.
POST	/api/alarms	Create alarm: label, when, days, timezone.
POST	/api/alarms/{id}/toggle	Pause or resume a future alarm.
POST	/api/alarms/{id}/snooze	Snooze a fired alarm for five minutes.
DELETE	/api/alarms/{id}	Delete an alarm definition.
DELETE	/api/history/{id}/{when}	Delete one history occurrence.
GET	/	Serve dashboard HTML; CSS and JS are served separately.

CHAPTER 6: IMPLEMENTATION

6.1 Native Server and Static Assets

main() reads the optional port and ALARM_DATA_DIR setting, creates the runtime directory, initialises the shared mapping and semaphore, loads existing alarm data, creates the event pipe, and forks the scheduler. The parent binds to 127.0.0.1, accepts a request, reads a bounded HTTP message and serves the allow-listed HTML, CSS and JavaScript files or JSON API responses.

6.2 Scheduler and Signals

The child installs a SIGALRM handler that sets a sig_atomic_t tick flag. It waits using sigsuspend() and, after each tick, checks all enabled records under the shared semaphore. One-time alarms are disabled when fired. Weekly alarms calculate their next matching local weekday in the alarm's stored time zone. A snoozed occurrence temporarily takes precedence over that next regular occurrence; after snooze fires, the recurring schedule remains intact.

6.3 Persistence, History and Snooze

The parent serialises alarm state to TSV after create, pause/resume, delete and fire/snooze operations. The append-only history stores each fired event, and a delete-history endpoint rewrites the history file through a temporary file before renaming it into place. A one-time snooze moves that alarm's due time five minutes forward. A recurring snooze sets a separate temporary deadline, preserving the weekly schedule.

6.4 Browser Logic

The browser keeps UI-only state for its timer, stopwatch, world-city selection, active ring panel and AudioContext. The dashboard refreshes its API state every two seconds and redraws alarm/history cards. An alarm-history event key combines alarm ID and occurrence timestamp so snoozing or a later weekly occurrence can be distinguished.

6.5 Security and Input Handling

The service binds only to loopback and limits static file routes to a fixed allow-list. The alarm API checks future timestamps, weekday-mask bounds, labels and time-zone identifiers. There is no user authentication because the service is intended only for local development; it should not be exposed on a public interface without a separate security redesign.

CHAPTER 7: FEATURES

7.1 Alarm Scheduling

A user opens New alarm, enters a name and a local date/time, then optionally selects one or more weekdays. Leaving all weekdays clear creates a one-time alarm. The browser submits a Unix timestamp, repeat mask and canonical local time-zone identifier; the Ubuntu scheduler checks the shared state and fires the event at the due time.

7.2 Ring, Stop and Snooze

When the dashboard observes a newly fired event, it shows the persistent ring alert and starts a repeating Web Audio tone. Stop sound ends the current browser tone. Snooze 5 min stops the ring and requests a five-minute snooze from the native service. Desktop notifications are separate and optional. If the browser is closed, the server can still schedule and record the event but cannot play browser audio.

7.3 Timer and Stopwatch

The timer supports minute/second entry, start, pause, resume and reset. At zero it enters DONE and joins the same ring/stop interface. The stopwatch uses the browser's monotonic performance clock, and supports start, pause, resume, lap capture and reset. These two tools are browser-local and do not persist across page reloads.

7.4 World Clock and History

The world-clock section contains selectable map markers, a searchable list of browser-supported time zones, live time and date cards, and each card's offset from the local time. Alarm history shows recent fired events. The × action deletes the selected history event without removing its alarm definition.

CHAPTER 8: TESTING

8.1 Test Strategy

The project includes a Python standard-library smoke test which starts the compiled executable on a free local port with an isolated temporary data directory. It calls the real HTTP service, checks returned assets and JSON, exercises one-time and weekly scheduling, waits for scheduler events, verifies snooze and restart persistence, and checks history deletion and its not-found response.

8.2 Test Cases and Results

Table 8.1 records the integration checks run against the Ubuntu build. The scheduler tests use near-future timestamps to make due events deterministic and quick to verify; they do not wait a full week for the next recurring occurrence.

Area	Verification	Result
Build	Clean GCC build with warning flags	PASS
Static assets	HTML, CSS and JavaScript served by local C server	PASS
Alarm CRUD	Create, pause, resume and delete API operations	PASS
One-time scheduling	Forked scheduler fires event and records history	PASS
Weekly recurrence	Selected local weekday fires; next date remains weekly	PASS
Snooze	Five-minute snooze works for one-time and recurring alarms	PASS
Persistence	Alarm, repeat mask, snooze and history survive parent restart	PASS
History deletion	Delete occurrence returns 200; second delete returns 404	PASS
Browser syntax	Node syntax check and live UI inspection	PASS

8.3 Build and Browser Checks

The C service was clean-built with GCC using -std=c11 -O2 -Wall -Wextra -Wpedantic. The smoke-test script passed on Ubuntu WSL. Node's syntax checker passed for the browser JavaScript. A local browser was also used to inspect the

dashboard, world-clock map and repeat-weekday alarm dialog. Browser UI screenshots are included in Appendix C.

8.4 Test Limitations

The smoke test is an integration test, not a formal safety certification or a long-duration reliability study. It does not measure sound level on each device, test operating-system suspend/resume behaviour, or validate every browser's notification permission flow.

CHAPTER 9: DEPLOYMENT

9.1 Ubuntu Installation

Install Ubuntu or Ubuntu on WSL 2. Install build-essential and python3. Open the project root in VS Code's WSL window. The project folder may reside at /mnt/c/Users/HP/OneDrive/Desktop/Os, although a folder under the Ubuntu home directory generally provides better Linux file-system performance.

- Build: make
- Run with defaults: make run
- Open http://localhost:8080 in a browser.
- Stop with Ctrl+C in the terminal or the VS Code Stop control.
- Test: make test

9.2 Direct VS Code Run

For a debugger-managed run, select Run and Debug, choose Chronos: Run in Ubuntu and press F5. The launch profile invokes the Chronos: build pre-launch task before running build/chronos from the project root. For a terminal-only run, choose Terminal > Run Task > Chronos: run. The server must be stopped before a new instance claims the same port.

9.3 Runtime Configuration

The default listen port is 8080; a different port can be passed as the executable's first argument. ALARM_DATA_DIR chooses where alarm/history files are written. The browser uses its local IANA time-zone setting. No API key, credential or remote endpoint is required.

CHAPTER 10: CHALLENGES AND LIMITATIONS

10.1 Challenges Encountered

Linux system headers, process APIs and signal interfaces are not native Windows interfaces. Opening the folder as a Windows workspace can therefore make IntelliSense report errors even though the source builds correctly in Ubuntu. A WSL VS Code window with Ubuntu GCC resolves that development-environment mismatch.

Browser audio is subject to autoplay policy. The page attempts to unlock its AudioContext after a user gesture, and the ring alert remains visible with stop/snooze controls if audio cannot start. Weekly schedules also require using the time zone captured when an alarm is created so weekday and daylight-saving transitions are interpreted locally.

10.2 Current Limitations

- Alarm sound and desktop notifications are browser-side; closing the page prevents audible output.
- Sleep or suspend may delay the process scheduler until the operating system resumes.
- The HTTP server is a small, single-request-at-a-time development server and is not production hardened.
- There is no account system, remote access, encryption or multi-user authorization.
- Timer, stopwatch and chosen world-clock cities are not stored across browser reloads.
- The world map shows a curated set of representative cities; the search list uses browser time-zone identifiers rather than every locality.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Improvements

- Add Linux desktop audio/notification support so alarms can ring when the browser is closed.
- Use a production HTTP library, robust JSON parser and a concurrent request model.
- Replace TSV storage with SQLite transactions and versioned migrations.
- Add browser-level automated tests and more scheduler fault-injection tests.
- Persist timer presets and selected world-clock cities.
- Add a service manager unit and graceful recovery after restart.

11.2 Possible Optimisations

A min-heap of upcoming due times could avoid scanning all alarm slots if the 64-alarm limit were raised. A request queue or multi-process/threaded server could improve concurrency, but would require extending the current synchronization and persistence model. Such changes are deliberately outside the small educational scope of this build.

CHAPTER 12: CONCLUSION

12.1 Summary

Chronos combines a native Ubuntu C/POSIX alarm service with a browser-based clock and time-management dashboard. It supports saved one-time and weekly alarms, snooze, firing history, a stoppable repeating browser tone, a timer, stopwatch, world clocks and VS Code/WSL workflows.

12.2 Achievements

The project demonstrates the full path from browser action to HTTP request, shared process state, POSIX scheduling and a persisted event. The working implementation builds on Ubuntu and passes its repeatable integration smoke test, including recurrence, snooze, service restart and history deletion.

12.3 Learning Outcomes

The implementation provides practice with C and POSIX process APIs, asynchronous signals, shared memory, semaphores, pipes, sockets, HTTP and JSON, browser state, time-zone handling, audio policy, file persistence, WSL tooling, test automation and technical documentation.

CHAPTER 13: REFERENCES

13.1 Technical References

The following primary documentation was consulted for the interfaces used in the project:

- The Open Group Base Specifications, POSIX process, signal, semaphore, memory-mapping and socket interfaces: https://pubs.opengroup.org/onlinepubs/9799919799/
- Linux man-pages project: fork(2), sigaction(2), sigsuspend(2), mmap(2), sem_init(3), poll(2), socket(2): https://man7.org/linux/man-pages/
- GNU Compiler Collection documentation: https://gcc.gnu.org/onlinedocs/
- GNU Make manual: https://www.gnu.org/software/make/manual/
- Microsoft Learn, Develop in Windows Subsystem for Linux with Visual Studio Code: https://code.visualstudio.com/docs/remote/wsl
- MDN Web Docs, Intl.DateTimeFormat: https://developer.mozilla.org/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat
- MDN Web Docs, Web Audio API: https://developer.mozilla.org/docs/Web/API/Web_Audio_API
- MDN Web Docs, Notifications API: https://developer.mozilla.org/docs/Web/API/Notifications_API

13.2 Project Materials

Implementation-specific behaviour and test details in this report were checked against the Chronos source tree, Makefile, VS Code workspace configuration and smoke test included with this project.

CHAPTER 14: APPENDICES

14.1 Quick User Guide

- Open the project in VS Code connected to WSL: Ubuntu.
- Use Run and Debug > Chronos: Run in Ubuntu, or run the Chronos: run task.
- Open http://localhost:8080.
- Create an alarm; select repeat weekdays if desired. An empty weekday selection makes a one-time alarm.
- Leave the browser dashboard open for sound; use Stop sound or Snooze 5 min when the ring panel appears.
- Use the alarm card toggle and delete button to pause, resume or remove schedules.
- Run make test from the Ubuntu terminal to execute the integration test.

14.2 API Endpoint Summary

The server exposes a small local API. Error responses include JSON messages and an appropriate HTTP status.

14.3 Project File Structure

The project separates the native process service, browser assets, runtime storage, tests and VS Code/Ubuntu configuration.

14.4 Representative Code

The scheduler waits for POSIX timer signals and inspects shared state; the browser sends alarm details through the same-origin JSON interface.

14.5 Interface Screenshots

The figures below are screenshots of the running project captured during documentation preparation. They illustrate the live dashboard, city map and weekday-repeat dialog.

HTTP API Reference

Method	Route	Purpose
GET	/api/state	Current alarm definitions and latest 20 history rows.
POST	/api/alarms	Create alarm: label, when, days, timezone.
POST	/api/alarms/{id}/toggle	Pause or resume a future alarm.
POST	/api/alarms/{id}/snooze	Snooze a fired alarm for five minutes.

DELETE	/api/alarms/{id}	Delete an alarm definition.
DELETE	/api/history/{id}/{when}	Delete one history occurrence.
GET	/	Serve dashboard HTML; CSS and JS are served separately.

Source Tree

```
Os/
  src/main.c                C/POSIX HTTP service and scheduler
  public/index.html         Dashboard markup
  public/style.css          Responsive pink interface
  public/app.js             Browser logic and API client
  data/                     Runtime alarm and history files
  tests/smoke_test.py       HTTP integration test
  .vscode/                  Ubuntu tasks and F5 launch profile
  Makefile                  Build, run, test and clean
```

Representative Scheduler Code

```
sigaction(SIGALRM, &action, NULL);
alarm(1);
while (keep_running) {
    sigsuspend(&wait_mask);
    if (!scheduler_tick) continue;
    scheduler_tick = 0;
    /* inspect shared alarms under the process-shared semaphore */
}
```

Interface Screenshots

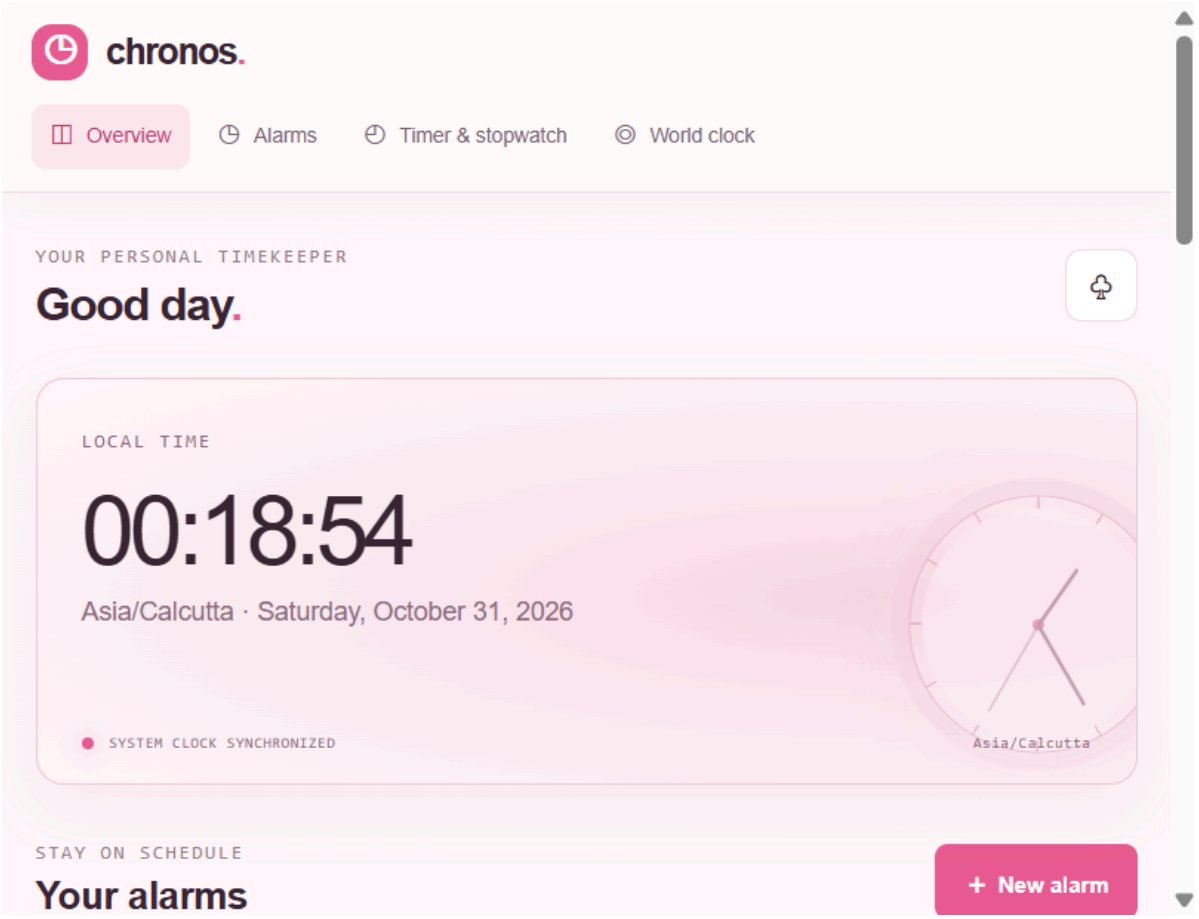


Figure 14.1: Chronos dashboard overview.

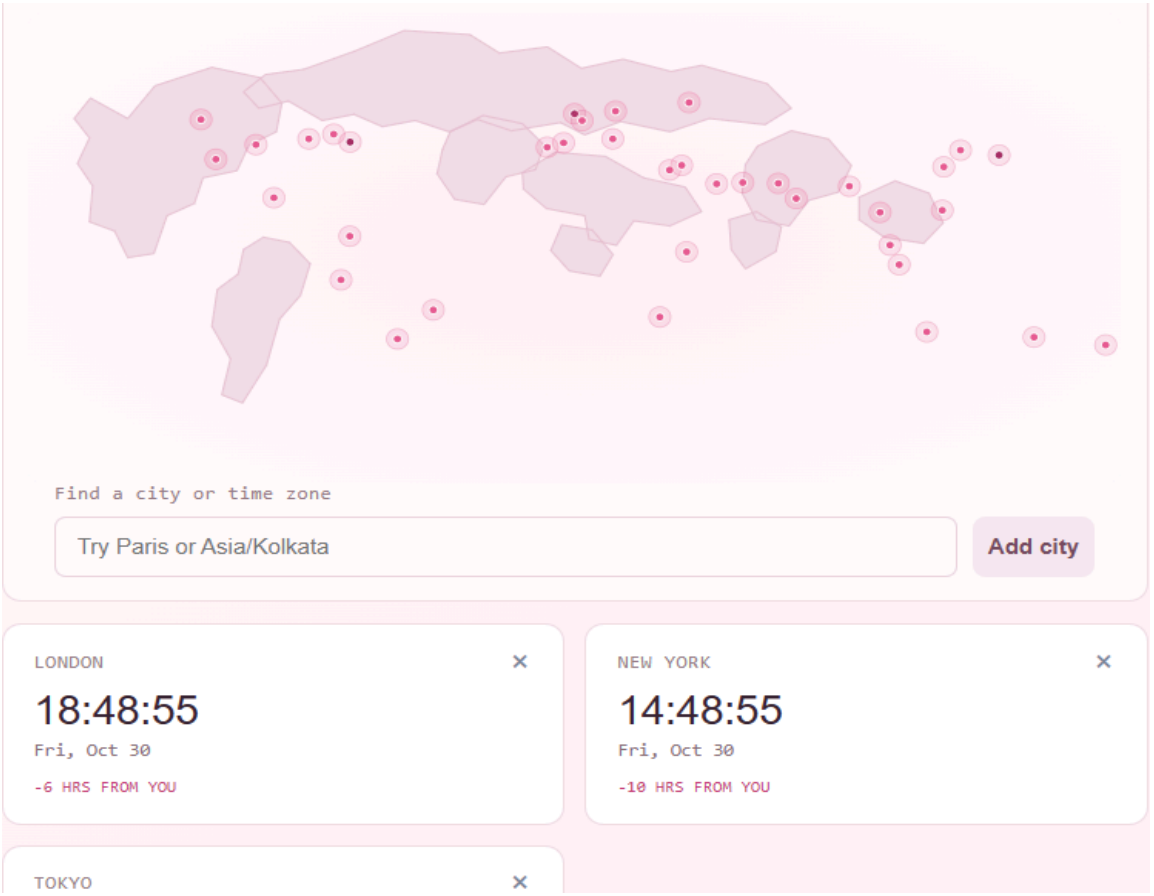


Figure 14.2: World-clock map and city cards.

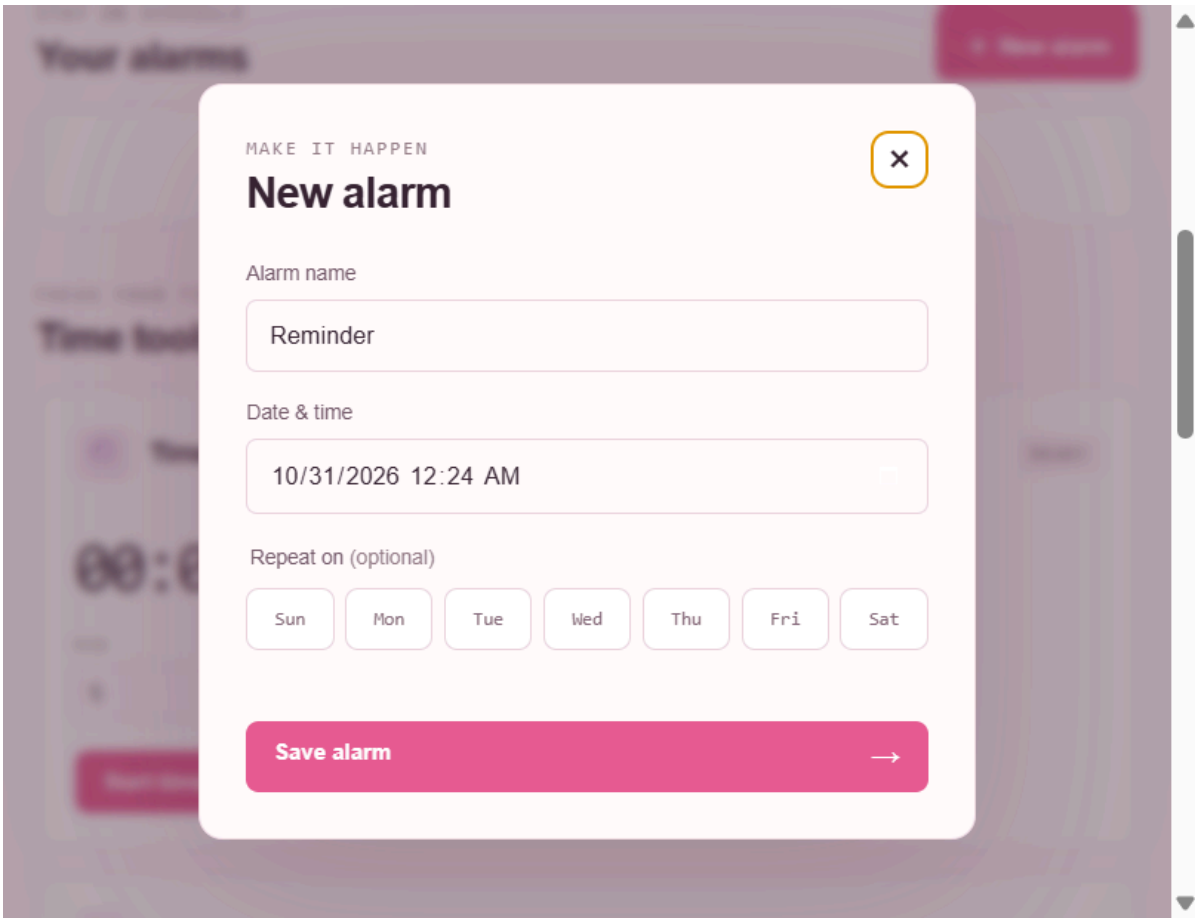


Figure 14.3: Alarm creation with weekday selection.